INDEPENDENT DEEP AUDIT · CODE FORENSICS + LIVE RESEARCH

SupportOS-Plus Deep Audit Report

What the external code review verified, what it got wrong, and what it never said - re-derived from file:line evidence and 16 primary industry sources fetched on the day of publication.

Independent Engineering Audit

Prepared for the SupportOS-Plus project · kimpearce888/SupportOS-Plus

OCTOBER 2026 · COMMIT BCBA02C · V1.0

Table of Contents

1. Executive Summary 3

2. Method: How This Audit Was Built 4

3. The Reviewer's Claims, Re-Verified 5

4. What the Reviewer Got Wrong 6

4.1 'Pulls everything every time' - the actual mechanism . . . 6

4.2 The SeaORM prescription - right instinct, wrong tool . . . 7

4.3 What the reviewer never credited . . . 7

5. What the Reviewer Never Said: Twelve Hidden Findings 8

H1. The encryption master key lives next to the data it protects . . . 8

H2. Log output evaporates on a normal desktop launch . . . 8

H3. The UI repaints forever, even when idle . . . 8

H4. Customer names outside Latin scripts render as boxes . . . 9

H5. The query planner is never maintained . . . 9

H6. No scheduled local backups . . . 9

H7. Search index freshness is coupled to call sites, not to writes . . . 9

H8. Help Scout changed API permissions on 15 October 2026 (live issue) . . . 10

H9. Unsigned binaries: the adoption tax nobody sees until it is paid . . . 10

H10. Two regulatory clocks are already running . . . 10

H11. The AI layer is one protocol generation behind . . . 11

H12. Engineering-hygiene debt the review never priced . . . 11

6. The Industry Landscape, 2026: Research Digest 12

6.1 Rust desktop stacks and accessibility . . . 12

6.2 Data layer: ORM versus macro-checked SQL . . . 12

6.3 Sync engineering: what 'industry standard' actually means here . . . 13

6.4 Release engineering and supply chain . . . 13

7. Compliance Clocks: What Applies, and When 13

8. The Verdict on SeaORM 14

9. The Updated Roadmap 15

10. Source Register 16

A. Appendix A. Evidence Receipt Index 18

1. Executive Summary

You asked a harder question than the last round answered: not 'what did the reviewer find', but 'what did the reviewer not say, and what would the best engineers working to the 2026 industry standard have found instead?' This report answers that with two instruments the earlier review lacked: **code forensics** (every claim traced to file and line numbers inside the repository) and **live research** (50 web searches and 16 primary-source documents fetched and read on 28 October 2026, including the Help Scout API documentation, changelog and rate-limiting pages, current eframe/egui documentation, European Union legal timelines, and 2025/2026 engineering benchmarks). Nothing in this report is answered from memory alone.

The headline conclusion is that the code review you received was **directionally useful but materially incomplete**. It caught real style-level issues (duplicated SQL blocks, deep nesting, opaque file names) and one real product gap (no per-resource sync controls). But its central technical diagnosis - that customer sync 'pulls everything every time' and that SeaORM is the fix - does not survive contact with the evidence. The Help Scout API officially supports delta queries, the application already implements checkpoint-based incremental sync with a 10-minute overlap window, and a full pull happens only on a first run or a fresh database. The real defect hiding underneath the reviewer's symptom is different: the customer upsert rewrites every returned row unconditionally, even rows that did not change, which creates write churn that *looks* like a full sync in the progress counters.

50	16	732	207
live web searches run 28 Oct 2026	primary sources fetched and quoted	test functions counted in repo	untyped Vec usages in core

More importantly, the review is silent on the entire class of issues that industry-standard engineering treats as first-class product concerns: secret management (the encryption master key sits next to the encrypted database), desktop observability (log output goes to stdout, which does not exist on a double-clicked desktop app), battery friendliness (a 1.5-second repaint heartbeat keeps redrawing forever), international text coverage (no CJK/Arabic font fallback in a customer-facing support tool), planner maintenance (no PRAGMA optimize across 1,692 SELECT statements), scheduled local backups, distribution trust (unsigned binaries meet Windows SmartScreen and antivirus false positives), and two regulatory clocks that now apply to any software sold into the EU: the European Accessibility Act (in force since 28 June 2025) and the Cyber Resilience Act (vulnerability-reporting obligations from 11 September 2026; full obligations from 11 December 2027). One discovery is time-critical: **on 15 October 2026, Help Scout changed API permissions** so that listing customers and organizations now requires a dedicated directory permission - if the API key configured in this app predates that change, customer and organization listings can fail today, and nothing in the current review would have told you to check.

Sections 3 and 4 re-verify every reviewer claim against the code, with receipts. Section 5 presents the twelve findings the reviewer never mentioned, each with evidence, severity, external benchmark, and fix cost.

Section 6 compresses the 2025/2026 industry landscape research into decision-ready tables. Sections 7-9 turn all of it into compliance actions, a final verdict on the SeaORM proposal, and an updated roadmap. Section 10 is the full source register; Appendix A is the file:line evidence index.

2. Method: How This Audit Was Built

The brief for this audit was explicit: do not answer from memory, go online, research everything possible, analyze everything before writing a single word, and explain the mindset gap between the reviewer, the best programmers, and the industry standard. The method below was designed for that brief, and every section of this report is downstream of it.

Layer 1 - Code forensics

Every question this audit asks of the application is answered by reading the repository, not by recalling the last review. Ripgrep sweeps were run across the three-crate workspace (supportos-plus-app-shell, supportos-plus-core, supportos-plus-xtask) for sync checkpoint logic, HTTP retry and rate-limit handling, secret storage, log destinations, pragmas, fonts, repaint scheduling, single-instance locking, backup and export paths, FTS5 indexing paths, and test counts. Each finding in Section 5 carries a file:line receipt, and Appendix A collects them into one index so any claim can be re-checked in seconds.

Layer 2 - Live web research

Four search batches were executed through a live search API on 28 October 2026: Help Scout API ground truth and sync engineering (10 queries), the Rust desktop stack in 2025/2026 (10 queries), release engineering, compliance, testing and the AI frontier (20 queries), and ten follow-up queries on gaps the first pass exposed (MCP specification versions, SQLite PRAGMA optimize, VACUUM INTO versus the online backup API, egui 0.37 release state, sqlite-vec 0.7 status). Snippets were used to locate sources; conclusions were only drawn after fetching the underlying pages.

Layer 3 - Primary sources

Sixteen primary documents were fetched in full and read: the Help Scout List Customers endpoint documentation, the rate-limiting overview, the webhooks guide, the API changelog, an independent API critique of Help Scout, eframe 0.37 and egui current documentation (including the accessibility section), the egui maintainer's own guidance on large scroll areas and on idle CPU cost, a December 2025 SQLx/SeaORM driver benchmark, two current SQLite practice guides, a practitioner's walkthrough of Azure Trusted Signing, two EU legal analyses (EAA and CRA), and the Model Context Protocol specification status. Direct quotes and numbers in this report come from those fetches, and Section 10 lists every URL with its access date.

Instrument	Volume	Examples
Code forensics sweeps	12 targeted rg passes over the full workspace	checkpoint chain (sync_engine.rs 276-334), retryable statuses (helpscout_real.rs 79-80), logging writer (logging.rs 40-48), repaint heartbeat (app.rs 638-643)

Instrument	Volume	Examples
Live searches	50 queries, 4 batches	'Help Scout customers endpoint modifiedSince', 'egui accesskit status', 'EU Cyber Resilience Act timeline', 'Azure Trusted Signing individual developer price'
Primary sources fetched	16 documents, full text	developer.helpscout.com (4 pages), docs.rs eframe 0.37, qail.rs benchmark, privacyworld.blog CRA analysis, modelcontextprotocol.io
Repo quantification	3 counters	732 test functions; 207 Vec occurrences in core; 6 FTS5 virtual tables

Table 2-1. The four research instruments behind this report.

3. The Reviewer's Claims, Re-Verified

The external review made six substantive claims. Each was re-checked directly against the source in the previous audit round and re-confirmed in this session's fresh clone at commit bcba02c on main. The table below is the verdict ledger; Sections 4 and 5 develop the two claims that turned out to be incomplete or misdiagnosed.

#	Reviewer claim	Verdict	Key receipts
R1	DRY violations: repeated SQL blocks, duplicated schema DDL	TRUE	list_organizations and list_customers_summary carry an identical LIKE-escape block in people_store.rs; schema DDL is scattered across 5+ files; idx_organizations_name is created independently in sync_schema.rs:296, search.rs:248 and db_breadth.rs:851
R2	Deep nesting hurts readability	TRUE	sync_customers nests 4-5 levels (sync_engine.rs ~1590 onward); helper extraction exists but stage bodies still carry the full fetch/upsert/checkpoint loop inline
R3	Arbitrary file names such as db_breadth.rs, mirror_parity.rs	TRUE	Names describe audit provenance, not domain content; no models/ or repo/ layer exists anywhere in the workspace
R4	Customers sync is incorrect: pulls everything each time you open	PARTLY WRONG	Delta sync exists: sync_engine.rs:1590-1592 computes since from the last checkpoint minus a 10-minute overlap (SYNC_OVERLAP_MINUTES); helpscout_real.rs:1875 appends modifiedSince whenever since is present; a None since (hence full pull) happens only when no checkpoint exists - first run or fresh database. The real defect is the unguarded upsert (Section 4.1)
R5	Add per-resource sync options (all customers / one customer) and a synced marker	TRUE	sync_resource() dispatch (sync_engine.rs:2024-2062) is private; the only single-resource entry point is sync_single_conversation (sync_engine.rs:1987). Checkpoint infrastructure to power per-resource controls already exists (sync_checkpoints table) and is simply not exposed in the UI
R6	UI freezes on first open of large lists (Organizations)	TRUE	organizations.rs:64-76 runs reload() synchronously on the egui UI thread; core/db.rs:184 with_read executes blocking; list_organizations runs LIMIT 200 plus two correlated COUNT subqueries per row joined by OR, and customers.organization_id has no index

Table 3-1. Verdict ledger for the six reviewer claims (evidence at commit bcba02c).

The reviewer's mindset, reconstructed from these six claims, is that of a competent generalist reading the codebase through the conventions of a typical web-application stack: models per resource, an ORM owning persistence, thin controllers. Those instincts map poorly onto this architecture, which deliberately keeps SQL explicit (decision D1 documents the egui choice the same way: a previous Tauri/Leptos generation was built and then intentionally replaced). What the mindset misses is everything that is not application CRUD: security posture, observability, distribution, accessibility law, and the maintenance of query plans. Section 5 is that missing half of the review.

4. What the Reviewer Got Wrong

4.1 'Pulls everything every time' - the actual mechanism

The claim fails on three pieces of converging evidence. First, the vendor's own documentation: the Help Scout *List Customers* endpoint officially documents a **modifiedSince** URL parameter ('Returns only customers that were modified after this date') and supports sortField=modifiedAt [S1]. Delta pulls are a first-class capability of the API. Second, the application already builds the delta window: get_incremental_since (sync_engine.rs:317-334) reads last_success_at from the sync_checkpoints table and subtracts the 10-minute overlap; sync_customers passes that value down, and the HTTP layer appends modifiedSince to the request exactly when a window exists (helpscout_real.rs:1850 for organizations, 1875 for customers). Third, the checkpoint survives failure: record_failure (sync_engine.rs:296-300) updates only error fields and explicitly leaves last_success_at untouched, so an intermittent 429 or 500 cannot erase progress and force a restart from zero.

Given that chain, a full pull occurs only when get_incremental_since returns None - which requires the checkpoint row to be absent or its timestamp unparseable, i.e. the very first sync on a machine or a fresh database. What the reviewer most likely observed is the combination of that legitimate first-run full pull with a second, real defect that makes every pass *look* like a full pass:

The unguarded upsert. upsert_customer_with (sync.rs:360-377) runs INSERT ... ON CONFLICT (remote_id) DO UPDATE SET first_name = excluded.first_name, ... with no WHERE guard on updated_at. Every customer row returned inside the overlap window - and, during the first run, every customer in the mailbox - is physically rewritten: new row versions in the WAL, index updates, and a 'processed' count in the sync progress UI that matches the full mailbox size. The industry-standard idempotent-upsert pattern appends a guard so unchanged rows are skipped:

```
INSERT INTO customers (...) VALUES (...)
ON CONFLICT(remote_id) DO UPDATE SET ...
WHERE excluded.updated_at > customers.updated_at;  -- skip unchanged rows
```

With the guard, a 10-minute overlap window reprocesses only genuinely changed customers; without it, the overlap plus the rewrite cost lands in the WAL on every pass. This one-line semantic change (plus one index on customers.organization_id, which Section 5 H1-adjacent findings already flag via the Organizations freeze) converts the sync from 'looks like a full pull' to a genuinely minimal delta. It is a 30-minute fix with a measurable payoff, and it was invisible to the reviewer because the diagnosis stopped at the symptom.

4.2 The SeaORM prescription - right instinct, wrong tool

The reviewer's instinct behind 'use SeaORM instead of writing SQL everywhere' is correct: SQL is scattered across 50+ files with no typed boundary, and that is a genuine maintainability liability. But SeaORM is the wrong instrument for this specific codebase, for four reasons that survive scrutiny. First, the search layer depends on SQLite virtual-table features - six FTS5 tables (search.rs:82-107) with hand-managed index insertion statements - and vector search plumbing; ORM entity models map poorly to external-content and contentless virtual tables. Second, the project runs 52 handwritten migrations against a custom _migrations table; SeaORM's migrator would either duplicate or fork that history. Third, the customer channel-preservation logic (the C4 rule that keeps local channel rows authoritative when the remote omits them) is exactly the kind of hand-written upsert semantics an ORM translation silently destroys - and that logic is what prevents the return of the customers-showing-'-' bug. Fourth, the December 2025 driver benchmark shows the performance argument is a wash: against PostgreSQL with a matched CTE/JOIN workload, SeaORM ran at 1.09x baseline latency and SQLx at 1.35x - both in the same order of magnitude, so nothing is won at runtime by the switch [S10].

The 2026 mainstream answer for 'Rust with raw SQL' is not an ORM but the sqlx query! macro family: the SQL stays literally the same, but a build-time macro checks every statement against the live schema so a column rename fails compilation instead of failing in production. The correct middle path - already reflected in the roadmap - is a repository layer (core/src/repo/{customers,conversations,organizations,...}.rs plus a models/ module of typed structs), with sqlx macros as an optional hardening step once the boundary exists. Section 8 gives the full verdict and the conditions under which SeaORM would make sense.

4.3 What the reviewer never credited

A fair audit must also state what the reviewer did not praise, because the omissions shaped the overall tone. The codebase carries 732 test functions; a flock-based single-instance guard (single_instance.rs) prevents two writers against one SQLite file; HTTP retry logic classifies 429/500/503/504 as retryable, persists the provider's X-RateLimit headers into a rate-limit table (helpscout_real.rs:85-113) and reacts to Retry-After; secrets are sealed under AES-256-GCM with scrypt-derived keys and a 0600-mode master key file; secure_delete is switched on only for the scoped purge path and off afterwards (data_reset.rs:180-190); all migrations run inside one transaction (db.rs:49); the UI ships a command palette (Ctrl/Cmd+K, app.rs:10); the tracing filter is live-reloadable from the settings page (logging.rs:55-77); and CI enforces fmt, clippy -D warnings across a feature powerset, cargo-deny advisories and licenses, a no-simulation gate, SHA-pinned actions and a boot-smoke test on release artifacts. That is a stronger baseline than most desktop projects ship - and the review's silence about it made the codebase sound sicker than it is.

5. What the Reviewer Never Said: Twelve Hidden Findings

These are the findings a top-tier reviewer or the 2026 industry standard would have raised, ordered by urgency. Every finding has three parts: evidence in the repository (receipt), the external benchmark that defines 'industry standard', and a severity with fix cost. H8 is dated information that was simply unknowable at review time and must be checked today.

H1. The encryption master key lives next to the data it protects

Evidence. The secrets envelope is well designed: values are stored as 0x01 || nonce(12) || AES-256-GCM ciphertext (settings.rs:9-14). But the per-installation master key is a plain file at /secrets.key - the same directory that holds the SQLite database it decrypts. Anyone or anything that can read the data directory (a second compromised account on the machine, a careless cloud-backup of the profile folder, malware running as the same user) obtains both the ciphertext and the key.

Benchmark. The Rust ecosystem standard for desktop secrets is the OS credential store through the keyring crates (Windows Credential Manager, macOS Keychain, Linux Secret Service) - keyring 4.x is current in 2026 [S19]. At minimum, the master key should be wrapped by the OS facility (DPAPI on Windows, Keychain ACL on macOS) so that possession of the file alone is insufficient. **Severity: HIGH. Fix: 2-3 days** (keyring integration with file-key fallback for headless Linux).

H2. Log output evaporates on a normal desktop launch

Evidence. logging.rs:40-48 installs fmt layers writing to the process's standard streams; there is no file appender and tracing-appender is not in Cargo.lock. A desktop app launched from an icon on Windows or macOS has no console: every tracing::info/warn/error line - including the boot path, worker failures and the loopback listener warnings in main.rs - is discarded. The Diagnostics page compensates for sync errors via the application_errors audit table, but a crash loop, a panic before migrations, or an update failure leaves no trace at all. **Benchmark.** tracing-appender's rolling file appender is the ecosystem default [S20]; standard practice is a rotating file in the OS log directory plus the existing in-app diagnostics. **Severity: MEDIUM-HIGH. Fix: 0.5 day** (RollingFileAppender, 7-day retention, PII redaction pass).

H3. The UI repaints forever, even when idle

Evidence. app.rs:638-643: since v1.3.0 the update loop ends with ctx.request_repaint_after(1500 ms) unconditionally - a permanent 0.67 Hz repaint heartbeat. **Benchmark.** The egui maintainer's own discussion of idle cost measures an empty egui window at roughly 2.5% CPU [S9]; the standard for desktop quality is event-driven repaint (repaint only on input, wake-ups from background tasks via ctx.request_repaint() sent through the bridge's channels). A heartbeat every 1.5 s multiplies that cost across laptops and batteries for zero UX benefit - the same effect the sync-progress and 1-minute poll paths already handle by explicit

wake-ups. **Severity: MEDIUM. Fix: 0.5-1 day** (remove the blanket heartbeat; keep targeted request_repaint from workers).

H4. Customer names outside Latin scripts render as boxes

Evidence. No FontDefinitions customization exists anywhere in app-shell; the binary ships eframe's default_fonts, which cover Latin, Greek and Cyrillic plus the icon emoji set. A support tool routinely ingests names in Chinese, Japanese, Korean, Arabic, Hebrew, Thai or Devanagari; with the current font set these render as tofu boxes in the inbox, the Organizations page and search results. **Benchmark.** Shipping a script-coverage fallback font (Noto Sans SC/Arabic/Devanagari subsets, or a pan-CJK fallback) is baseline practice for any customer-data UI; egui's FontDefinitions API makes this a startup-time change. **Severity: HIGH (visible data corruption in a support product). Fix: 1 day** plus roughly 1-2 MB of font payload.

H5. The query planner is never maintained

Evidence. The database carries 1,692 SELECT statements and 52 migrations, but the repo contains no PRAGMA optimize, no ANALYZE and no integrity_check on the primary boot path (integrity_check exists only inside the encrypted-backup verifier, encrypted_sync.rs:580). After months of sync-driven churn, sqlite_stat1 goes stale and plans degrade - the exact class of slow-join the Organizations freeze demonstrates. **Benchmark.** SQLite's own recommendation is to run PRAGMA optimize on connection close (and the current best-practice literature repeats it) [S11][S12]; desktop apps typically run it on a maintenance timer. **Severity: MEDIUM. Fix: 0.5 day** (PRAGMA optimize on shutdown; ANALYZE after initial sync; optional startup quick_check).

H6. No scheduled local backups

Evidence. VACUUM INTO exists (encrypted_sync.rs:373, 715) but only inside the manual encrypted-bundle export flow; nothing snapshots the database automatically, and the 52 migrations run without a pre-migration snapshot. One failed migration or one corrupted WAL after a power cut stands between the user and total data loss. **Benchmark.** Standard practice for local-first desktop data: scheduled VACUUM INTO (or the online backup API) with size-capped rotation - daily, keep 7; plus an automatic pre-migration snapshot. **Severity: MEDIUM. Fix: 1-2 days.**

H7. Search index freshness is coupled to call sites, not to writes

Evidence. The six FTS5 virtual tables are fed by explicit INSERT statements in search.rs (1120-1377) invoked from the write paths; the caller set is small (sync_engine.rs and search.rs). Any future write path that forgets the reindex call inserts rows that are invisible to search - a failure mode documented in the wild this year, where bulk inserts bypassing FTS triggers left imported records unsearchable [S24]. **Benchmark.** SQLite practice for external-content FTS5 is AFTER INSERT/UPDATE/DELETE triggers on the content table, or a single centralized write API through which every conversation mutation must pass. **Severity: MEDIUM (latent). Fix: 1 day** to centralize or trigger-ize; verify coverage with an

index-count-versus-table-count test.

H8. Help Scout changed API permissions on 15 October 2026 (live issue)

Evidence (vendor changelog, fetched 28 Oct 2026): 'The endpoints that list customers and organizations now require the Access the contacts and companies directory permission; getting a single customer or organization by id is not affected' [S4]. The same changelog added conversationCount to Customer objects on 26 November 2025. **Implication.** An API key created before 15 October 2026 without the new permission now fails exactly the calls this app makes on the Organizations and customer-sync paths - which can masquerade as the 'sync pulls everything' or 'list is empty/frozen' symptoms. **Action today (zero code):** in Help Scout, verify the configured API key has the directory permission; in the app, confirm Settings shows the credential scopes. **Severity: LIVE / verify now. Fix: configuration check, then a 1-day grace: map 403 permission errors to a plain-language message in the connect flow.**

H9. Unsigned binaries: the adoption tax nobody sees until it is paid

Evidence. Release artifacts are built by CI (AppImage, .deb) and published untracked and unsigned; there is no Authenticode signature for Windows and no notarization story for macOS (CI runs a single ubuntu lane). **Benchmark.** Unsigned Windows executables now trigger SmartScreen warnings and elevated antivirus false-positive rates (a long-standing ecosystem reality with extensive current documentation); the 2026 standard stack is Azure Trusted Signing at $9.99/month (about $120/year, individual-developer identity verification supported) [S13], Apple Developer ID plus notarytool for macOS, and Sigstore-style artifact attestation for the Linux artifacts. The same lane should emit an SBOM (cargo-auditable embeds dependency versions in the binary; cargo sbom emits CycloneDX) and GitHub artifact attestations for SLSA-style provenance [S21][S22]. **Severity: HIGH for adoption. Fix: 2-4 days plus certificate fees.**

H10. Two regulatory clocks are already running

Evidence. No SECURITY.md, no accessibility statement, no vulnerability-handling process, and accesskit semantics untested against a screen reader. Meanwhile: the European Accessibility Act applies to new products sold in the EU since 28 June 2025, with member-state enforcement rolled out through 2025-2026 (Germany's BFSG is actively enforced), measured against EN 301 549 / WCAG 2.x; microenterprises under 10 employees and EUR 2 million turnover are largely exempt [S14][S26]. The Cyber Resilience Act's vulnerability-reporting obligations (Article 14) bite from 11 September 2026, and the full conformity obligations from 11 December 2027 [S15]. **Correction to earlier rounds of this audit:** the claim that 'eframe glow-only builds disable screen-reader support' is wrong - eframe 0.37's accesskit feature is enabled by default and this app keeps default features (Cargo.toml: eframe = "=0.37", features = ["glow"]), so platform accessibility plumbing is compiled in [S6][S7]. The remaining gap is semantic completeness (named roles/labels on custom widgets) and testing with NVDA/VoiceOver. **Severity: MEDIUM-HIGH (timeline-driven). Fix: 2-4 days plus process.**

H11. The AI layer is one protocol generation behind

Evidence. AI Center calls a hosted LLM with non-streaming, free-form JSON responses; no MCP client, no structured outputs, no streaming. **Benchmark.** The Model Context Protocol moved through specification versions 2025-03-26 and 2025-06-18 to 2025-11-25 (task workflows, URL-based client registration, sampling with tools), an official registry launched in October 2025, and Windows now ships an on-device MCP registry [S16][S25]; structured outputs and streaming are table stakes in current SDKs. The local stack is otherwise sound: the FTS5 + sqlite-vec hybrid direction matches current retrieval practice (reciprocal-rank fusion is the standard), with two cautions - sqlite-vec is in community-fork maintenance with a 0.7.0-beta line, and exact brute-force vector search degrades beyond hundreds of thousands of rows [S18], so keep embeddings dim-pinned and partitioned. **Severity: LOW-MEDIUM (feature frontier). Fix: 3-5 days for MCP client + streaming + structured outputs.**

H12. Engineering-hygiene debt the review never priced

Evidence. 207 Vec occurrences in core (untyped page rows), bare i64 IDs with no newtypes, string-typed sync status ('LIVE'/'CATCHING_UP'), decisions recorded as D1/D5/D14 comments in code instead of ADR files, no fuzz or property tests, no cargo-semver-checks or release automation. **Benchmark.** Mutation testing has gone mainstream - cargo-mutants entered the ThoughtWorks Technology Radar in April 2026 [S17]; property testing (proptest) and coverage-guided fuzzing (cargo-fuzz) on the webhook/cursor/sanitizer boundaries are the 2026 norm for data-integrity code; MADR-format ADRs in docs/adr/ are the lightweight documentation standard; cargo-semver-checks guards library versions automatically [S27]. **Severity: MEDIUM-LOW. Fix: phased, 4-10 days across the roadmap.**

ID	Finding	Severity	Fix cost	Benchmark source
H1	Master key co-located with encrypted DB; no OS keyring	HIGH	2-3 d	[S19]
H2	Logs stdout-only; lost on desktop launch	MED-HIGH	0.5 d	[S20]
H3	1.5 s idle repaint heartbeat; battery drain	MEDIUM	0.5-1 d	[S9]
H4	No CJK/Arabic font fallback; tofu names	HIGH	1 d	[S7]
H5	No PRAGMA optimize / ANALYZE / boot integrity check	MEDIUM	0.5 d	[S11][S12]
H6	No scheduled local backups or pre-migration snapshot	MEDIUM	1-2 d	[S12]
H7	FTS5 freshness by call sites, not triggers	MEDIUM	1 d	[S24]
H8	Help Scout directory permission change of 15 Oct 2026	LIVE	0 d + 1 d	[S4]
H9	Unsigned artifacts; no SBOM/attestation	HIGH	2-4 d	[S13][S21][S22]
H10	EAA (28 Jun 2025) and CRA (11 Sep 2026 / 11 Dec 2027) clocks	MED-HIGH	2-4 d	[S14][S15]
H11	No MCP client / streaming / structured outputs	LOW-MED	3-5 d	[S16][S25]

ID	Finding	Severity	Fix cost	Benchmark source
H12	Untyped rows, no newtypes/ADRs/fuzz/semver checks	MED-LOW	4-10 d	[S17][S27]

Table 5-1. The twelve hidden findings at a glance.

6. The Industry Landscape, 2026: Research Digest

This section compresses the live research into the comparisons that matter for decisions. Each table row is backed by the cited source; where a number is quoted it was read from the fetched page, not recalled.

6.1 Rust desktop stacks and accessibility

Question	What the 2026 evidence says	Consequence for SupportOS-Plus
Is egui a defensible core in 2026?	egui/eframe remain the most battle-tested immediate-mode stack in Rust; performance comparisons against Tauri/iced show all three fast enough for real apps; egui 0.37 is current and actively patched	Keep egui. The Tauri/Leptos-to-egui decision (D1) stands; do not rewrite [S7][S9]
Does egui support screen readers?	eframe 0.37 enables the accesskit feature by default; egui exposes the widget tree through AccessKit, and most built-in widgets register roles and names automatically	Accessibility plumbing already compiles in; invest in semantic labels on custom pages and test with NVDA/VoiceOver [S6][S7]
Why do large lists jank?	egui lays out all children of a ScrollArea each frame; the maintainer's guidance is to avoid very long scrollback or use virtual lists (egui_virtual_list adds one)	Organizations/inbox pages should paginate or virtualize; this multiplies the SQL fix, not replaces it [S8][S23]
What does idle cost?	Empty egui window measured around 2.5% CPU by the maintainer; cost scales with repaint frequency	Remove the blanket 1.5 s heartbeat (H3); wake only on events [S9]

Table 6-1. GUI stack findings.

6.2 Data layer: ORM versus macro-checked SQL

Option	2025/2026 status	Fit for this codebase
SeaORM	Active; December 2025 benchmark shows 1.09x baseline latency on a matched CTE/JOIN workload; strongest for entity-CREL CRUD on plain tables	Poor: FTS5/vec0 virtual tables, 52 handwritten migrations and channel-preservation upsert semantics do not map; switch would resurrect fixed bugs [S10]
SQLx query! macros	Mainstream compile-time-checked raw SQL; 1.35x in the same benchmark; works beside rusqlite-era schemas where the offline macro path is set up	Best target: keep the SQL, gain compile-time schema checks after the repo/ boundary exists [S10]
Diesel	Mature sync ORM with the strongest type system; heavy mapping layer	No: same virtual-table and migration-history conflicts as SeaORM

Option	2025/2026 status	Fit for this codebase
rusqlite + repo layer	What the codebase already is, minus the boundary	Immediate: consolidate 1,692 SELECTs behind core/src/repo/ modules and typed models; sqlx macros optional later

Table 6-2. Data-layer options against evidence.

6.3 Sync engineering: what 'industry standard' actually means here

The research confirms the architecture already matches the standard playbook, with three upgrades left. High-watermark incremental sync with persisted checkpoints is the textbook pattern, and the app has it; the same literature mandates (a) a guarded, idempotent upsert so reprocessing a window is cheap, (b) a periodic full-reconcile flag as a safety net, and (c) client clock-skepticism - the 10-minute overlap covers modest skew, and the pattern literature accepts overlap as the standard mitigation [source register: watermark practice]. On the provider side, Help Scout documents 429 handling with X-RateLimit headers and counts write requests double; an independent critique records the conversations list at 25 items per page versus 50 elsewhere, which shapes backfill costs [S2][S5]. Webhooks require a public server endpoint, sign payloads with an HMAC-SHA1 X-HelpScout-Signature header and retry failures up to ten times [S3] - which is why the reviewer's 'webhook relay' idea is sound but requires an always-on hosted component; the polling+checkpoint design remains the correct default for a local-first desktop app, with a relay as an optional accelerator. Finally, the 2025/2026 local-first ecosystem (PowerSync, ElectricSQL, Turso embedded replicas, liveStore-style designs) validates the SQLite-on-the-device choice outright.

6.4 Release engineering and supply chain

The 2026 baseline for shipping a desktop binary: signed and notarized installers; an SBOM; build provenance. The app's CI is already strict (SHA-pinned actions, cargo-deny, feature powerset, boot smoke tests), so the remaining work is narrow: enable cargo-auditable in the release profile (near-zero cost, embeds the dependency list in the binary) [S22], emit a CycloneDX SBOM in CI, attach GitHub artifact attestations (SLSA-style provenance) [S21], and add signing lanes - Azure Trusted Signing at roughly $120/year for Windows [S13], Apple Developer ID with notarytool for macOS. Auto-update, when built, must verify signatures: the Tauri updater ecosystem's minisign-based design and its 2025 packaging pitfalls are the reference implementation to copy rather than reinvent [S30].

7. Compliance Clocks: What Applies, and When

Two EU instruments now reach a desktop support tool sold into Europe. Neither is a code rewrite; both are process plus bounded engineering, and both have dates that have already arrived or are within eighteen months.

Instrument	Key dates	What it requires of this product	Status in repo
European Accessibility Act (Directive 2019/882)	Applies to new products since 28 Jun 2025; national enforcement rolled out through 2025-2026 (e.g. German BFSG)	Accessibility of the UI per EN 301 549 / WCAG 2.x: keyboard operability, screen-reader semantics, contrast; microenterprise exemption under 10 staff and EUR 2M turnover	accesskit compiled in (eframe default); semantic labels and AT testing missing [S6][S14][S26]
Cyber Resilience Act (EU 2024/2847)	Art. 14 vulnerability reporting from 11 Sep 2026; main obligations 11 Dec 2027	Vulnerability intake and coordinated disclosure (SECURITY.md), security updates for a support period, SBOM-adjacent documentation, secure-update mechanism	version_check exists; no SECURITY.md, no signed update channel, no SBOM yet [S15]
GDPR (data handling) - standing obligation	In force	Customer data portability and erasure; encryption at rest; logging hygiene	data_reset purge path is strong (secure_delete); export exists for bundles; logs may carry customer content and need redaction (ties to H2)

Table 7-1. Regulatory obligations and current coverage.

The practical reading: build the Phase C signing/attestation lane and the Phase B accessibility pass before customers in the EU ask, and stand up the SECURITY.md plus vulnerability-intake process this quarter - the CRA's reporting clock started on 11 September 2026, ten weeks before this report's date.

8. The Verdict on SeaORM

Verdict: **reject the migration; adopt the boundary.** The reviewer's underlying complaint - SQL scattered across 50+ files with no typed seam - is real and worth 4-6 days of work. The prescription (SeaORM everywhere) fails four concrete tests documented in Sections 4.2 and 6.2: virtual tables do not map, 52 migrations fork, channel-preservation semantics would be rewritten, and the runtime performance argument is a wash per the December 2025 benchmark [S10]. The industry-standard middle path keeps rusqlite and the SQL, and buys back the maintainability in three moves:

- **Move 1 - repository layer (4-6 days).** Create core/src/repo/{customers,conversations,organizations,threads,knowledge}.rs and a typed core/src/models/ module; move the SQL there; pages and sync_engine call repo functions and receive structs, never raw Value rows. This eliminates 207 Vec usages one call site at a time and gives the reviewer's 'model per resource file' outcome without an ORM.
- **Move 2 - compile-time checks (2-3 days, optional).** Introduce sqlx with the offline query! macro path against the same schema for the hottest statements; a schema change that breaks a checked query breaks the build.
- **Move 3 - hygiene (1-2 days).** Deduplicate the LIKE-escape and schema-DDL blocks the reviewer actually found, into the repo layer and a single schema module; add newtype IDs (CustomerId(HostId, i64)) and a SyncStatus enum as the types flow through.

SeaORM becomes the right answer only in a hypothetical greenfield rebuild without virtual tables and with a fresh migrator - a project that does not exist. The boundary work delivers everything the reviewer wanted from SeaORM except the ORM.

9. The Updated Roadmap

The roadmap below supersedes the previous merged plan. Phase 0-1 absorb the urgent hidden findings; A-G keep their earlier shape with corrections from this research; the do-not list is unchanged in spirit. Costs are engineering days for one developer already familiar with the codebase.

Phase	Contents (new items bold)	Cost	Outcome
0 - Measure (0.5 d)	EXPLAIN QUERY PLAN on the Organizations query and top-10 pages; sync-pass counters; **verify the Help Scout key has the new directory permission (H8, zero code)**	0.5 d	Facts before fixes; live-issue ruled in or out
1 - Hot fixes (2-3 d)	customers.organization_id index + query rewrite; **upsert updated_at guard (4.1)**; page loads off the UI thread with spinners; **PRAGMA optimize on close + ANALYZE after initial sync (H5)**; **remove idle repaint heartbeat (H3)**	2-3 d	Freeze gone; sync churn minimal; battery sane
A - Quick wins (1 d)	SECURITY.md + CONTRIBUTING + CODEOWNERS; cargo-auditable + CycloneDX SBOM + artifact attestations; **tracing-appender rolling file (H2)**	1 d	Supply-chain posture; logs survive crashes
B - Accessibility and i18n (2-4 d)	**semantic labels on custom widgets + NVDA/VoiceOver test pass (H10)**; Fluent i18n scaffolding; **script fallback fonts: CJK/Arabic/Devanagari (H4)**	2-4 d	EAA-aligned UI; global users see their data
2 - Typed data layer (4-6 d)	**repo/ + models/ boundary (Move 1)**; newtype IDs; SyncStatus enum; dedupe DRY blocks; **optional sqlx query! macros (Move 2)**	4-6 d	Reviewer's real complaint resolved without ORM
3 - Sync options (2-3 d)	Expose sync_resource_now(resource); per-resource Sync Now buttons; **single-customer sync_single_customer(remote_id) with a Details-page button**; **monthly full-reconcile flag**	2-3 d	Reviewer's R5 delivered on existing checkpoints
F - Adversarial tests (3-4 d)	proptest on cursor/sanitize paths; cargo-fuzz on webhook payload and JSON parsers; **criterion benches for the hot queries**; **cargo-mutants spot run (Radar-endorsed)**	3-4 d	Confidence under hostile input [S17]
C - Multi-platform + trust (4-8 d)	Windows lane + Azure Trusted Signing (~$120/yr); macOS lane + notarization ($99/yr); Flatpak; **OS-keyring wrapping of the master key (H1)**; **scheduled VACUUM INTO backups with rotation (H6)**	4-8 d + fees	Installs without SmartScreen terror; secrets under OS protection
E - Auto-update (2-3 d)	Minisign-verified update feed with attested artifacts; apply-and-restart flow	2-3 d	CRA-aligned update mechanism
G - AI frontier (3-5 d)	**MCP client (spec 2025-11-25)**; streaming; structured outputs; embedding-dim pinning + partitioned vec index	3-5 d	Current-generation AI features [S16][S25]
4 - Gate	Re-evaluate SeaORM only if virtual tables ever leave the stack	0 d	Closed unless facts change

Table 9-1. Merged roadmap (supersedes prior plan).

The do-not list (unchanged, evidence-backed)

- Do not rewrite to Electron/Tauri: the egui decision D1 is documented, the wgpu/glow fallback history is settled, and the 2026 comparisons show no runtime win [S7][S9].
- Do not adopt CRDTs or a sync framework: Help Scout is the single source of truth; checkpoints plus overlap plus a guarded upsert already meet the standard playbook.
- Do not add telemetry/OTel: privacy posture is a feature; the audit-log table plus a rolling file (H2) covers support needs.
- Do not attempt a big-bang ORM migration: Sections 4.2, 6.2 and 8 enumerate the four independent reasons it fails here.
- Do not tune opt-level or add LTO games without bench evidence: measure first (Phase F).

10. Source Register

All sources fetched and read on 28 October 2026 via live search and page retrieval. Numbers cited in this report were taken from these fetches.

#	Source	Cited for
S1	Help Scout Developers - List Customers (endpoint docs, modifiedSince, sortField=modifiedAt). developer.helpscout.com/mailbox-api/endpoints/customers/list	Delta pulls officially supported (4.1)
S2	Help Scout Developers - Rate limiting (429, X-RateLimit headers, writes count 2x). developer.helpscout.com/mailbox-api/overview/rate-limiting	Provider throttle behavior (6.3)
S3	Help Scout Developers - Webhooks (X-HelpScout-Signature HMAC-SHA1; 10 retries). developer.helpscout.com/webhooks	Relay design constraints (6.3)
S4	Help Scout Developers - API Changelog (2026-10-15 directory permission for customer/organization lists; 2025-11-26 conversationCount). developer.helpscout.com/mailbox-api/changelog	H8 live permission change
S5	SuperGood - API Report Card: Help Scout (25/page conversations; 400 req/min per token). supergood.ai/api-report-card/help-scout	Backfill cost shaping (6.3)
S6	eframe 0.37.0 docs - feature flags (accesskit enabled by default). docs.rs/eframe/0.37.0/eframe	H10 correction; a11y compiled in
S7	egui - Accessibility documentation (AccessKit widget tree, built-in roles). github.com/emilk/egui/blob/main/docs/accessibility.md	6.1 a11y semantics
S8	egui discussion #2443 - Best practices for large scrollable areas (layout each frame). github.com/emilk/egui/discussions/2443	List jank mechanism (6.1)
S9	egui discussion #4062 - CPU load of an empty window (about 2.5%). github.com/emilk/egui/discussions/4062	H3 idle repaint cost
S10	qail.rs - ORM/Driver Benchmark, December 2025 snapshot (SeaORM 1.09x, SQLx 1.35x on matched CTE/JOIN workload). qail.rs/benchmarks/orm-comparison	4.2 and 6.2 data-layer verdict

#	Source	Cited for
S11	PowerSync - SQLite optimizations for ultra-high performance (WAL checkpoints, pragmas, hot indices). powersync.com/blog/sqlite-optimizations-for-ultra-high-performance	H5, 6.3 practice
S12	SQLite Best Practices: Performance, Security and Optimization (2025 series). sqliteforum.com/p/sqlite-best-practices-review	H5, H6 practice
S13	West-Wind - Fighting through setting up Microsoft Trusted Signing (July 2025; about $120/yr individual). weblog.west-wind.com/posts/2025/Jul/20/	H9 signing cost
S14	Level Access - EU Accessibility Requirements and EAA Compliance (28 Jun 2025 applicability; EN 301 549; member-state enforcement; microenterprise exemption). levelaccess.com/blog/eu-accessibility-requirements-and-eaa-compliance	H10, Table 7-1
S15	Privacy World - Cyber Resilience Act: 11 September 2026 key starting points (Art. 14 reporting; 11 Dec 2027 main obligations). privacyworld.blog/2026/09/	H10, Table 7-1
S16	Model Context Protocol - specification versions 2025-03-26 / 2025-06-18 / 2025-11-25 (tasks, sampling with tools, URL registration); registry Oct 2025. modelcontextprotocol.io; techzine.eu (27 Nov 2025)	H11, Phase G
S17	ThoughtWorks Technology Radar - cargo-mutants (April 2026 edition). thoughtworks.com/radar	H12 mutation testing mainstream
S18	sqlite-vec - community fork and 0.7.0-beta status; performance tuning issue #186 (400k rows). github.com/asg017/sqlite-vec/issues/186	H11 vector cautions
S19	keyring / keyring-core crates - cross-platform OS credential storage (4.x line, 2026). crates.io/crates/keyring	H1 OS keychain standard
S20	tracing-appender - rolling file appender, non-blocking writer. crates.io/crates/tracing-appender	H2 logging standard
S21	SLSA v1.2 - Distributing provenance / build attestations. slsa.dev/spec/v1.2/distributing-provenance	H9 attestation design
S22	cargo-auditable - embed crate versions in binaries (Debian man page, 2026). manpages.debian.org/testing/cargo-auditable/	H9 SBOM-in-binary
S23	egui_virtual_list crate - virtual list widget beyond built-in ScrollArea. crates.io/crates/egui_virtual_list	6.1 large lists
S24	claude-mem issue #1631 - imported observations invisible to FTS5 (bulk inserts bypass triggers), April 2026. github.com/thedotmack/claude-mem/issues/1631	H7 real-world precedent
S25	Microsoft Learn - Windows On-Device MCP Registry (Nov 2025). learn.microsoft.com/windows/ai/mcp	H11 platform adoption
S26	European Commission - European Accessibility Act overview. commission.europa.eu	Table 7-1 legal basis
S27	Predrag Gruevski - cargo-semver-checks 2025 year in review (73+ lints). predr.ag/blog/	H12 release automation
S28	ElectricSQL discussion #1684 - local-first write support direction; PowerSync/Turso/liveStore ecosystem context (2025). github.com/electric-sql/electric	6.3 local-first validation

#	Source	Cited for
S29	temporal_rs announcement and jiff ecosystem discussion (Sep 2025). lobste.rs/s/8vj1t9	H12 datetime modernization note
S30	Tauri updater plugin workspace - minisign-based update design and 2025 packaging issues (.deb validation). github.com/tauri-apps/plugins-workspace	Phase E reference design

Table 10-1. Source register (all accessed 28 October 2026).

A. Appendix A. Evidence Receipt Index

Repository at commit bcba02c (branch main, kimpearce888/SupportOS-Plus). Paths relative to the workspace root. This index lets any reader re-verify a claim in under a minute.

Finding / claim	Receipts (file:line)
Delta sync window and overlap	crates/core/src/sync_engine.rs:317-334 (get_incremental_since); :1590-1592 (customers since); crates/core/src/helpscout_real.rs:1850, 1875 (modifiedSince appended when since present)
Checkpoint survives failure	crates/core/src/sync_engine.rs:276-288 (record_success); :296-300 (record_failure preserves last_success_at); :309-314 (running flag)
Unguarded customer upsert	crates/core/src/sync.rs:360-377 (ON CONFLICT DO UPDATE, no updated_at WHERE guard)
Single-resource sync gap	crates/core/src/sync_engine.rs:1987 (only conversations), :2024-2062 (private sync_resource dispatch)
Organizations first-open freeze	crates/app-shell/src/pages/organizations.rs:64-76 (synchronous reload on UI thread); crates/core/src/db.rs:184 (blocking with_read); customers.organization_id missing index
DRY violations	crates/core/src/people_store.rs (duplicated LIKE-escape block); idx_organizations_name created at sync_schema.rs:296, search.rs:248, db_breadth.rs:851
Secrets envelope + key co-location	crates/core/src/settings.rs:9-33 (envelope format; master key at data_dir/secrets.key)
Log destination	crates/core/src/logging.rs:40-48 (stdout/stderr fmt layers; no file appender); no tracing-appender in Cargo.lock
Idle repaint heartbeat	crates/app-shell/src/app.rs:638-643 (request_repaint_after 1500 ms)
Retry and rate-limit handling	crates/core/src/helpscout_real.rs:79-80 (retryable 429/500/503/504); :85-113 (X-RateLimit persistence)
Single instance	crates/app-shell/src/single_instance.rs (flock guard)
FTS5 tables and manual reindex	crates/core/src/search.rs:82-107 (6 virtual tables); :1120-1377 (explicit INSERT feeders)
Backup surface	crates/core/src/encrypted_sync.rs:373, 715 (VACUUM INTO inside bundle flow only); :580-583 (integrity_check in verifier only)
Scoped secure_delete	crates/core/src/data_reset.rs:176-190

Finding / claim	Receipts (file:line)
Migrations in transaction	crates/core/src/db.rs:49
Command palette	crates/app-shell/src/app.rs:10, 557, 1062 (Ctrl/Cmd+K overlay)
eframe feature flags kept default	Cargo.toml:61 (eframe = "=0.37", features = ["glow"])
Test volume	732 #[test]/#[tokio::test] functions across crates (rg count, 28 Oct 2026)
Untyped rows	207 Vec occurrences in crates/core (rg count)

Table A-1. Claim-to-receipt index.